

Ejemplo Práctico: Editor de Matriz CSV

En este ejemplo vamos a aplicar los conceptos de interfaces gráficas para resolver un problema real: **Leer, mostrar y modificar un archivo .csv** que contiene una matriz simétrica (AxA) de números enteros.

A continuación, destacaremos los hitos más importantes de esta implementación.

1. Código Modular (Separando Lógica e Interfaz)

Si revisas los archivos, notarás que el programa está dividido en dos partes:

- **gestor_matriz.py**: Es la **lógica de negocio**. Se encarga *exclusivamente* de abrir el archivo, leerlo, validar las reglas (simetría y enteros) y guardar. ¡No sabe que existe Tkinter!
- **main.py**: Es la **interfaz gráfica**. Se encarga *exclusivamente* de dibujar botones, organizar las celdas y recibir los clics del usuario.

¿Cuáles son las ventajas del Código Modular?

1. **Reutilización**: Si el día de mañana el profesor pide hacer este programa en consola (sin Tkinter), puedes usar **gestor_matriz.py** exactamente igual, sin cambiarle ni una sola línea.
2. **Fácil mantenimiento**: Si hay un error visual (un botón mal puesto), sabes que está en **main.py**. Si hay un problema matemático o de validación de datos, sabes que está en **gestor_matriz.py**.
3. **Trabajo en equipo**: En un proyecto grande, un estudiante puede programar el diseño de la ventana y otro puede programar cómo se leen los archivos, trabajando simultáneamente sin chocar.

El Uso de Constantes Globales (RUTA_CSV)

Si observas **main.py**, verás que la ruta del archivo está guardada en una variable (escrita en mayúsculas por convención) justo al principio del código:

```
RUTA_CSV = "matriz.csv"
```

¿Por qué esto es vital para mantener el orden? Imagina que usaras el texto **"matriz.csv"** directamente en 10 partes distintas de tu código. Si mañana te piden que el archivo se llame **"datos_finales.csv"**, ¡tendrías que buscar y modificar ese texto línea por línea, arriesgándote a olvidar alguna o cometer un error de tipeo! Al usar una constante al inicio, si necesitas cambiar el archivo a leer, solo modificas **una única línea de código** y todo el programa se actualiza de inmediato. Te ahorrará incontables dolores de cabeza en proyectos grandes.

NOTA: Este tipo de variables se suelen colocar en un archivo llamado **.env** (donde se almacena información sensible o de configuración del sistema), luego leídas por el programa al momento de ejecutarse. Para efectos de este ramo basta y sobra que sean ordenados y utilicen constantes globales.

2. Lectura y Validación de Archivos

En `gestor_matriz.py` no solo leemos datos, sino que usamos **Excepciones** (`raise`) para atrapar posibles escenarios donde el usuario se equivoque:

- Utilizamos la librería `os` para verificar si existe el archivo (`os.path.exists()`). Si no existe, lanzamos un `FileNotFoundError`.
- Transformamos los strings en números usando `int()`. Si falla (porque pusieron letras), capturamos el `ValueError` y lanzamos nuestro propio mensaje personalizado.
- Contamos el largo de las filas vs la cantidad de filas para validar la simetría matemática ($A \times A$).

Todos estos errores "viajan" hacia la interfaz gráfica que es la encargada de avisarle al humano.

¿Por qué usamos `raise` y cómo funciona?

En programación modular, el archivo de lógica (`gestor_matriz.py`) **nunca debe** mostrar ventanas de Tkinter ni usar `print()`. Su único trabajo es procesar los datos de forma invisible.

Si la lógica detecta un dato corrupto (ej: una letra en vez de un número), usa `raise ValueError("Mensaje")` para **"lanzar"** la alerta. Es un grito de auxilio que dice *"¡No puedo continuar!"*. Luego, la interfaz (`main.py`) que llamó a la función se encarga de **"atrapar"** ese grito utilizando un bloque `try / except` y decide mostrárselo al usuario de forma bonita.

Modelo Visual del Flujo de Errores:

```
[ Usuario ] -> Clic en "Cargar" -> [ main.py (Interfaz) ]
                                   | (1. Llama a cargar_matriz())
                                   v
                                   [ gestor_matriz.py (Lógica) ]
                                   (2. Lee el CSV y encuentra una letra
'A')
                                   |
                                   | (3. Lanza el error: raise
ValueError)
                                   v
[ Usuario ] <- Muestra MessageBox <- [ main.py (Interfaz) ]
                                   (4. Atrapa el error con 'except
ValueError')
```

3. Manejo de Ventanas de Texto (MessageBox)

En `main.py` utilizamos la sublibrería `tkinter.messagebox`. Es la manera estándar y más amigable de entregar *feedback* (retroalimentación) al usuario mediante ventanas emergentes. Los tres casos principales son:

- `messagebox.showerror(titulo, mensaje)`: Muestra un ícono de error rojo. Lo usamos cuando la operación falla rotundamente (ej: no existe el archivo CSV, o el usuario intentó guardar la letra "X" en vez de un número).
- `messagebox.showwarning(titulo, mensaje)`: Muestra un ícono de advertencia amarillo. Lo usamos para problemas de formato, como cuando la matriz tiene 3 filas y 4 columnas (no es

simétrica).

- `messagebox.showinfo(titulo, mensaje)`: Muestra un ícono informativo azul. Lo usamos para indicarle al usuario operaciones exitosas, como "Matriz cargada correctamente" o "Cambios guardados con éxito".

🔧 ¡Pruébalo tú mismo! (Desafío)

Intenta interactuar con el programa provocando intencionalmente los errores para ver las cajas de alerta:

1. Abre `matriz.csv` con un bloc de notas y escribe letras en lugar de números. Ejecuta `main.py` e intenta cargar.
2. Abre `matriz.csv` y añade un número más al final de la primera fila para romper la simetría (AxB). Intenta cargar.
3. Elimina o cámbiale el nombre a `matriz.csv` e intenta presionar "Cargar".
4. Carga la matriz correctamente, pero luego en los recuadros blancos de la ventana, cambia un número por una letra e intenta hacer clic en "Guardar Cambios".